Ruby has some other print tools that do a better job with more complex structured data. There is the jj method, which you need to require json to have access to, and which creates pretty-printed JSON. It also has y, which comes when you require yaml and produces the argument in YAML syntax. We also recommend the awesome-print gem, which when loaded and required, gives you ap, a method that produces a very clear structure of complex data.

You should find one of these tools useful when writing output; it's often faster to put one of these statements in, say, a test and run the test and inspect the output, then it is to use a step debugger.

You can use Ruby's reflection abilities quite powerfully for printing information to the console. The caller method is part of Kernel and will always show you the current call stack. Aaron Patterson has a lot of helpful output tips at https://tenderlovemaking.com/2016/02/05/i-am-a-puts-debuggerer.html.

The Ruby Debugger

Debuggers can also be quite powerful, and Ruby 3.1 came with a big improvement to Ruby's standard debugger. The new debugger is distributed as a separate Ruby gem, rather than part of the standard library:

```
$ gem install debug
```

Note that the gem is named *debug* and not *debugger*. The *debugger* gem is a separate tool that hasn't been maintained since 2015. Similarly, if you are using bundler, specify a version constraint of ">= 1.0.0" or else you will get a different, older gem. Global, permanent namespaces are fun!

After you have installed the debugger gem, there are a few different ways to start the debugger.

You can start your script with the command rdbg rather than ruby, as in:

```
rdbg code/trouble/profileeg.rb
```

The script will start in debug mode, which we'll talk about in a moment. If you want to run the debugger on a Ruby process that's invoked via a separate command like rake, then you use rdbg -c --, as in:

```
rdbg -c -- rake test:load.
```

More commonly, you'd use the debugger by inserting the line binding.break into your code (you must have done a require "debug" somewhere along the code path). The debugger is invoked at that line.

Visual Studio Code also has a plugin[1] that allows you to invoke the new Ruby debugger using VS Code's existing debugging interface. (To be clear, many editors support Ruby debugging but may do so via a different tool.)

You can also use the debugger remotely against an application that is not running in your terminal (often because it's running in a Docker container, or is a background process, or is not using the standard in and out ports). To connect to a remote process, you start with rdbg

1. https://marketplace.visualstudio.com/items?itemName=KoichiSasada.vscode-rdbg